



Universidad de Cuenca

Facultad de Ingeniería
Ingeniería en Telecomunicaciones

Microprocesadores, Microcontroladores y Sistemas Embebidos

Kenneth S. Palacio Baus
kenneth.palacio@ucuenca.edu.ec

Jaime Patricio Chiqui Chiqui
jpatricio.chiquic@ucuenca.edu.ec

Práctica 3

PIC18F4550 - Dispositivos de Visualización: Multiplexación de Displays 7 segmentos

- **Valoración:** 10 puntos.
- **Tipo de Trabajo:** Trabajo práctico e informe individual.
- **Objetivos:** Mediante la presente práctica el estudiante aprenderá a:
 - Visualizar información de carácter numérico en un conjunto de displays de 7 segmentos.
 - Utilizar la técnica de multiplexación de displays mediante interrupción del TMR0 para optimizar el hardware.
 - Realizar un dispositivo temporizador controlado.
- **Recursos:** Como base de esta práctica, utilizaremos MPLAB X IDE y la hoja de datos del microcontrolador PIC18F4550.

Instrucciones

Para obtener una calificación en la presente práctica, cada estudiante deberá entregar un informe escrito según la estructura que se menciona más adelante.

- Envíe su trabajo mediante la plataforma e-nova.
- El nombre el archivo de su informe debe tener el formato:
ApellidoNombre_Pract03_MicroCon_M26.pdf
- Si su envío no cumple con el nombre de archivo y fecha de entrega, no recibirá calificación.

1. Materiales y Equipos

Para el desarrollo de la presente práctica, se utilizaron los siguientes recursos y componentes electrónicos:

- **Software:** MPLAB X IDE v5.x, compilador XC8 C.
- **Microcontrolador:** PIC18F4550 de Microchip.
- **Displays:** 4 displays de 7 segmentos, ánodo común. Uso 4 displays en uno solo **5641AS**
- **Transistores:** 4 transistores NPN (2N3904) para control de displays.
- **Resistencias:**
 - 13 resistencias de 1 k (Displays + base de transistores).
 - 3 resistencias de 10 k (pull-up para pulsantes).
- **Oscilador:** Cristal de cuarzo de 20 MHz.
- **Capacitores:** 2 capacitores cerámicos de 22 pF (cristal).
- **Pulsantes:** 3 micropulsantes (S1, S2, S3).
- **LED:** 2 LED para indicador en RD0 y Alimentacion.
- **Programador:** PICkit 3 con cable USB.
- **Otros:** Protoboard, cables de conexión.

2. Marco Teórico

2.1. Display de 7 Segmentos

Un display de 7 segmentos es un dispositivo de visualización que permite representar dígitos del 0 al 9 mediante el encendido o apagado selectivo de 7 segmentos de LED. La disposición de estos segmentos sigue una nomenclatura estándar (a, b, c, d, e, f, g).

2.1.1. Estructura Física

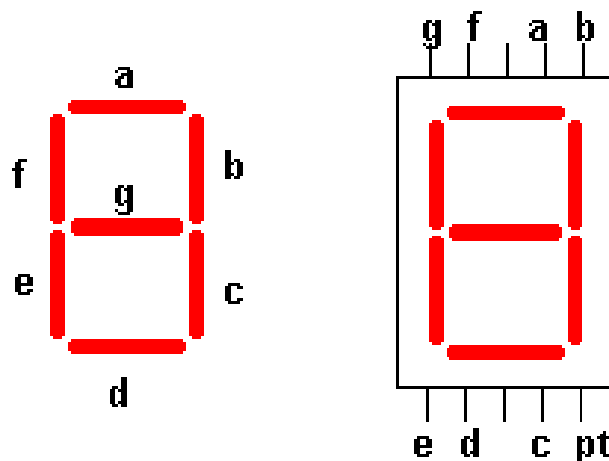


Figura 1: Nomenclatura de segmentos en display de 7 segmentos

2.1.2. Tipos de Displays

Existen dos configuraciones principales:

- **Ánodo Común:** Todos los ánodos están conectados a un punto común de voltaje (+5V). Se encienden llevando los segmentos a nivel bajo (0V).
- **Cátodo Común:** Todos los cátodos están conectados a tierra (GND). Se encienden llevando los segmentos a nivel alto (+5V).

2.1.3. Display Múltiple 5641AS (4 en 1)

El componente **5641AS** es un módulo integrado que agrupa cuatro displays de 7 segmentos en un solo encapsulado físico. Este tipo de display está diseñado específicamente para aprovechar la técnica de multiplexación, ya que internamente conecta en paralelo todos los segmentos homólogos de los cuatro dígitos (el segmento 'a' del dígito 1 con el 'a' del dígito 2, etc.).

- **Distribución de pines:** Cuenta con un total de 12 pines. Ocho de ellos corresponden a los segmentos (a, b, c, d, e, f, g) y al punto decimal (DP), mientras que los cuatro pines restantes son los terminales comunes (ánodo o cátodo) independientes para cada uno de los cuatro dígitos.
- **Ventaja principal:** Al compartir los buses de los segmentos, reduce la cantidad de conexiones necesarias hacia el microcontrolador de 32 pines (si fueran individuales) a tan solo 12 pines, facilitando enormemente el cableado en la protoboard.

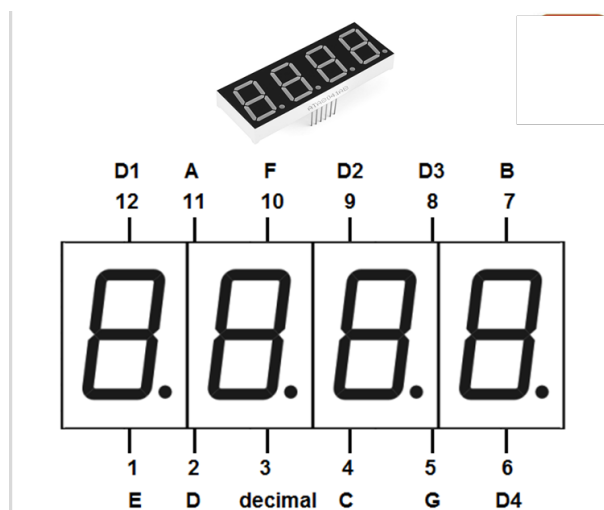


Figura 2: Módulo de display 7 segmentos 4 en 1 (Modelo 5641AS) y su distribución de pines

En esta práctica se utilizaron displays de catodo común, lo que significa que debemos enviar lógica (1 = encendido, 0 = apagado).

2.2. Técnica de Multiplexación

La multiplexación es una técnica que permite controlar múltiples displays usando menos pines del microcontrolador.

2.2.1. Principio de Funcionamiento

En lugar de mantener todos los displays encendidos simultáneamente de forma permanente, se encienden uno a la vez muy rápidamente. Si la frecuencia de conmutación es lo suficientemente alta (mayor a 50 Hz), el ojo humano percibe los cuatro displays encendidos al mismo tiempo debido a la persistencia de la visión. **Frecuencia de refresco necesaria:**

Una frecuencia de al menos 50 Hz es necesaria para una percepción libre de parpadeos. Para el caso específico de un sistema con **3 displays** (como el detallado en las notas de la Figura 3) y utilizando un tiempo por display de **2.5 ms** por interrupción, se obtiene una frecuencia de refresco de la siguiente manera:

- Tiempo de ciclo completo (T): $3 \text{ displays} \times 2.5 \text{ ms/display} = 7.5 \text{ ms}$
- Frecuencia de refresco total (f_R): $1 / T = 1 / 0.0075 \text{ s} \approx 133.3 \text{ Hz}$

Esta frecuencia es significativamente mayor que los 50 Hz mínimos, lo que garantiza una visualización libre de parpadeos. La Figura 3 muestra el patrón de señales para este caso, basado en las notas manuscritas proporcionadas.

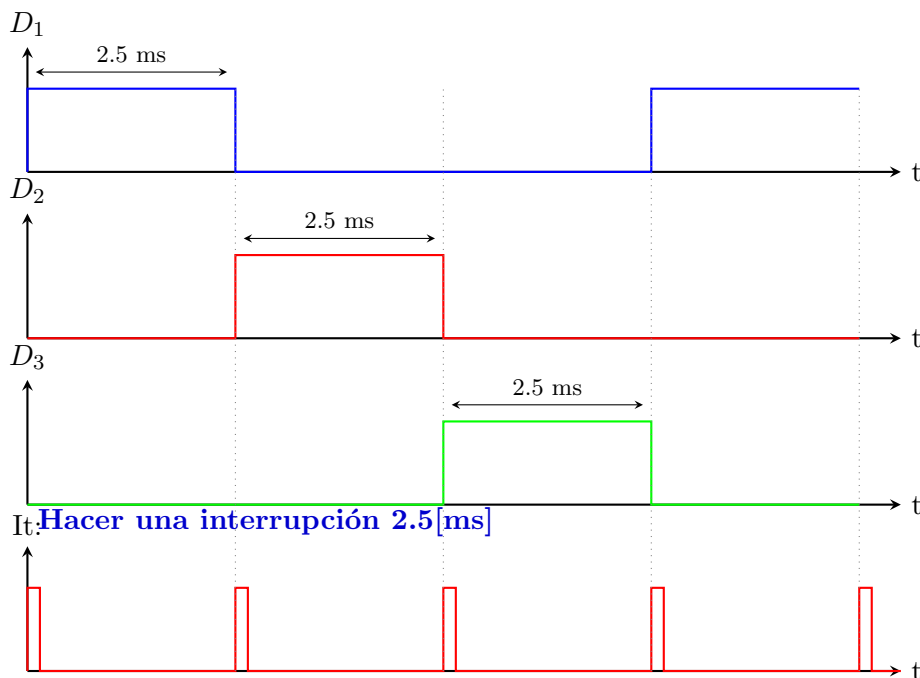


Figura 3: Patrón de multiplexación de 3 displays e interrupciones del temporizador cada 2.5 ms

2.2.2. Ventajas de la Multiplexación

- **Reducción de pines:** Se necesitan solo 7 pines para segmentos + 4 pines para select (11 total) en lugar de 32 pines si cada display estuviera independiente.
- **Reducción de consumo de potencia:** En promedio, cada display está encendido solo 1/4 del tiempo, reduciendo significativamente el consumo.
- **Mantenimiento simplificado:** El código es más modular y fácil de escalar a más displays.

2.3. Códigos de 7 Segmentos

Cada número o letra requiere un código binario específico que indica qué segmentos deben estar encendidos. Para el display 5641AS utilizado en esta práctica, se emplea una configuración

de **cátodo común**, lo que significa que los segmentos se encienden aplicando un nivel alto (1 lógico).

En el código implementado, el puerto se configura asumiendo el orden de bits [a b c d e f g dp], donde el bit más significativo (bit 7) corresponde al segmento *a* y el menos significativo (bit 0) corresponde al punto decimal (*dp*).

Cuadro 1: Códigos de 7 segmentos (Cátodo Común - activo en alto)

Dígito	a	b	c	d	e	f	g	Hex (con dp=0)
0	1	1	1	1	1	1	0	0xFC
1	0	1	1	0	0	0	0	0x60
2	1	1	0	1	1	0	1	0xDA
3	1	1	1	1	0	0	1	0xF2
4	0	1	1	0	0	1	1	0x66
5	1	0	1	1	0	1	1	0xB6
6	1	0	1	1	1	1	1	0xBE
7	1	1	1	0	0	0	0	0xE0
8	1	1	1	1	1	1	1	0xFE
9	1	1	1	1	0	1	1	0xF6
H	0	1	1	0	1	1	1	0x6E
o	0	0	1	1	1	0	1	0x3A
L	0	0	0	1	1	1	0	0x1C
A	1	1	1	0	1	1	1	0xEE

Nota: Los valores hexadecimales corresponden a la configuración directa de los puertos del microcontrolador, donde cada bit en "1" enciende un segmento. El bit 0 se mantiene en 0 para mantener apagado el punto decimal (dp), aunque en este diseño se utiliza una máscara para conservar el estado del pin RD0 (LED indicador).

2.4. Timer0 para Multiplexación

El Timer0 se configura para generar interrupciones a intervalos regulares (típicamente 10-50 ms) que controlan cuándo cambiar el display activo. Esta interrupción ocurre en segundo plano sin necesidad de intervención del programa principal.

Frecuencia de refresco necesaria:

Para que el efecto visual sea imperceptible:

- Mínimo 50 Hz para percepción sin parpadeo
- Con 4 displays: $50 \text{ Hz} \times 4 = 200 \text{ Hz}$ por display individual
- Tiempo por display: $1/200 \text{ Hz} = 5 \text{ ms}$

3.2. Funcionamiento del Software

3.2.1. Inicialización del Sistema

```
1 // --- CONFIGURACION CRITICA PARA USAR RC4 y RC5 ---
2 UCONbits.USBEN = 0; // Deshabilita el modulo USB
3 UCFGbits.UTRDIS = 1; // Convierte a E/S digital
4 // -----
5
6 // Configuracion de entrada / salida
7 TRISD = 0; // Puerto D todo salida
8 TRISCbits.TRISC0 = 0; // RC0=D4 salida
9 TRISCbits.TRISC1 = 0; // RC1=D3 salida
10 TRISCbits.TRISC6 = 0; // RC6=D2 salida
11 TRISCbits.TRISC7 = 0; // RC7=D1 salida
12 TRISC |= 0b00110000; // RC4=S2 y RC5=S3 entradas(en 1)
13
14 // OFF Display (evita ghosting: int. repentina)
15 LATD = 0;
16 LATCbits.LATC7 = 0; // D1
17 LATCbits.LATC6 = 0; // D2
18 LATCbits.LATC1 = 0; // D3
19 LATCbits.LATC0 = 0; // D4
20
21 // =====
22 // INICIALIZACION: RDO -> 10 pulsos de 50ms, luego ON
23 // =====
24 for(int i=0; i<10; i++){
25     LATDbits.LATD0 = !LATDbits.LATD0; // toggle
26     DelayXms(50);
27 }
28 LATDbits.LATD0 = 1; // ON Led
```

Listing 1: Función de inicialización (Fragmento del main)

3.2.2. Configuración del Timer0

```
1 // =====
2 // Configuracion de TMRO - MODO 16 BITS
3 //
4 // TOCON:
5 // bit7: TMR0ON = 0 (apagado por ahora)
6 // bit6: T08BIT = 0 (modo 16-bit)
7 // bit5: T0CS = 0 (reloj interno Fosc/4)
8 // bit4: T0SE = 0 (no importa en interno)
9 // bit3: PSA = 0 (prescaler activo)
10 // =====
11
12 // Prescaler 1:32 TOPS=100 (Caso para test)
13 TOCON = 0b00000100;
14
15 // Configuracion de interrupciones
16 RCONbits.IPEN = 1; // Prioridades habilitadas
17 INTCON2bits.TMR0IP = 1; // TMRO alta prioridad
18 INTCONbits.TMR0IE = 1; // Interrupcion TMRO ON
19 INTCONbits.TMR0IF = 0; // Limpiar bandera
20 INTCONbits.GIEH = 1; // Global alta ON
21 INTCONbits.GIEL = 1; // Global baja ON
22
23 TOCONbits.TMR0ON = 1; // Encender TMRO
```

Listing 2: Configuración del Timer0 (Fragmento del main)

3.2.3. Rutina de Servicio de Interrupción

```
1 void __interrupt(high_priority) InterrupcionHP(void){
2 // =====
3 // ISR ALTA PRIORIDAD - TMRO 16Bits
4 // =====
5     if(INTCONbits.TMROIF){
6         if(estado == 0){
7             // MODO S3: Carga 2.5ms (HoLA se ve solido y fijo sin parpadear)
8             TMROH = 0xF1; TMROL = 0x5A;
9         } else{
10             //=====
11             //             50 [ms]
12             //=====
13             TMROH = 0xB6; TMROL = 0x82;
14         }
15         // Evita superposicion
16         LATCbits.LATC7 = 0;
17         LATCbits.LATC6 = 0;
18         LATCbits.LATC1 = 0;
19         LATCbits.LATC0 = 0;
20
21         // Multiplexacion
22         if(visDisplay){
23             LATD = (datos[multiplex] & 0xFE) | (LATDbits.LATD0);
24             // 0xFE = 11111110 -> limpia bit0 (dp) pero conserva RD0 del LED
25
26             switch(multiplex){
27                 case 0: LATCbits.LATC7 = 1; break; // D1
28                 case 1: LATCbits.LATC6 = 1; break; // D2
29                 case 2: LATCbits.LATC1 = 1; break; // D3
30                 case 3: LATCbits.LATC0 = 1; break; // D4
31             }
32         } else {
33             LATD = 0x00 | (LATDbits.LATD0);
34         }
35         multiplex++;
36         if(multiplex > 3) multiplex = 0;
37
38         INTCONbits.TMROIF = 0;
39         return;
40     }
41 }
```

Listing 3: ISR para multiplexacion

3.2.4. Función de Multiplexación

```
1 void MostrarSecuencia(void){
2     int d, n;
3     // Recorre los 4 displays: 0=D1, 1=D2, 2=D3, 3=D4
4     for(d = 0; d < 4; d++){
5         for(n = 0; n <= 9; n++){
6             // Pone el numero en el display activo, el resto apagado
7             datos[0] = 0x00;
8             datos[1] = 0x00;
9             datos[2] = 0x00;
10            datos[3] = 0x00;
11            datos[d] = codigosNum[n]; // solo el display d muestra n
12            DelayXms(250); // 250ms por numero
13        }
14        // Display se apaga al terminar (datos[d] queda en 9, luego se limpia)
```



```

15     datos[d] = 0x00;
16     DelayXms(100);           // pausa apagado entre displays
17 }
18 }

```

Listing 4: Funcion de multiplexacion secuencial de displays

3.2.5. Cálculo del Intervalo de Multiplexación

Para que los displays no presenten un parpadeo molesto, es necesario que el ciclo completo de multiplexación sea lo suficientemente rápido como para engañar al ojo humano (persistencia de la visión). Dado que opero el PIC18F4550 a una frecuencia de 48 MHz (gracias al PLL), el tiempo de cada ciclo de instrucción es:

$$T_{cy} = \frac{4}{48 \text{ MHz}} = 83,33 \text{ ns}$$

A continuación, presento los cálculos de los valores de precarga (TMR0H y TMR0L) para los distintos tiempos de prueba solicitados en la práctica, utilizando los prescalers adecuados para que el temporizador de 16 bits no se desborde antes de tiempo:

Casos con Prescaler 1:8 ($T_{tick} = 83,33 \text{ ns} \times 8 = 666,67 \text{ ns}$):

- **Para 2.5 ms:**

Cuentas = $2,5 \text{ ms} / 666,67 \text{ ns} = 3750$

Valor de precarga = $65536 - 3750 = 61786 \rightarrow \mathbf{0xF15A}$

- **Para 5 ms:**

Cuentas = $5 \text{ ms} / 666,67 \text{ ns} = 7500$

Valor de precarga = $65536 - 7500 = 58036 \rightarrow \mathbf{0xE2B4}$

- **Para 10 ms:**

Cuentas = $10 \text{ ms} / 666,67 \text{ ns} = 15000$

Valor de precarga = $65536 - 15000 = 50536 \rightarrow \mathbf{0xC568}$

- **Para 25 ms:**

Cuentas = $25 \text{ ms} / 666,67 \text{ ns} = 37500$

Valor de precarga = $65536 - 37500 = 28036 \rightarrow \mathbf{0x6D84}$

Caso con Prescaler 1:32 ($T_{tick} = 83,33 \text{ ns} \times 32 = 2666,67 \text{ ns}$):

- **Para 50 ms:**

Cuentas = $50 \text{ ms} / 2666,67 \text{ ns} = 18750$

Valor de precarga = $65536 - 18750 = 46786 \rightarrow \mathbf{0xB682}$

4. Partes de la Práctica

4.1. Parte 1: Multiplexación de Displays

4.1.1. Descripción

En esta parte implementé la técnica de multiplexación para mostrar la palabra "HoLA" en los cuatro displays. Realicé pruebas físicas variando los tiempos de interrupción para cada display (desde 2.5 ms hasta 50 ms) con el objetivo de analizar el tiempo óptimo donde el parpadeo desaparece.

4.1.2. Funcionamiento

1. **Inicialización:** El LED RD0 parpadea 10 veces (50 ms cada ciclo), luego permanece encendido para confirmarme que el PIC arrancó correctamente.
2. **Display normal:** Los displays muestran la palabra "HoLA".
3. **Modo velocidad de prueba (S2):** Al presionar S2, el sistema comienza a usar el tiempo de interrupción configurado para evaluar el parpadeo.
4. **Modo fijo óptimo (S3):** Al presionar S3, se obliga a la interrupción a regresar a los 2.5 ms, mostrándome la palabra "HoLA" de forma estática y perfecta.

4.1.3. Tabla de Pruebas y Observaciones Personales

Durante las pruebas que realicé en el circuito físico, fui modificando el registro del Timer0 para evaluar los distintos retardos. Las observaciones reales que obtuve las resumo en la siguiente tabla:

Cuadro 2: Resultados visuales de las pruebas de multiplexación

Test	Tiempo por Display	Observaciones que realicé
1	2.5 ms	No hay ningún parpadeo. La palabra se ve completamente estática, brillante y perfecta.
2	5 ms	Tampoco noté parpadeo. La visualización se mantiene estable y la persistencia visual sigue siendo fija.
3	10 ms	Empieza a notarse un parpadeo muy ligero, pero es casi imperceptible a menos que preste mucha atención.
4	25 ms	Aquí el parpadeo ya es claramente notorio y la transición entre los displays se vuelve bastante evidente.
5	50 ms	Se ve bastante el parpadeo. Se nota perfectamente cómo se enciende y se apaga cada display uno por uno.

Conclusión: Al realizar estas pruebas, concluí que los tiempos de **2.5 ms** y **5 ms** son los únicos ideales para una visualización profesional en esta práctica. En mi código principal establecí 2.5 ms como el valor por defecto para el modo óptimo (S3).

4.2. Parte 2: Reloj Digital

4.2.1. Descripción

Se implementa un reloj digital que muestra horas y minutos en formato HH:MM. El usuario puede ajustar la hora mediante pulsantes.

4.2.2. Funcionamiento

Inicialización:

- Displays muestran "00:00"
- Displays parpadean a 1 Hz para indicar que la hora no está configurada
- LED RD0 parpadea a 1 Hz indicando funcionamiento

Ajuste de hora (S2):

- Presión única: incrementa los minutos en 1
- Presión sostenida: incremento rápido de minutos
- Displays dejan de parpadear una vez que se presiona S2

Decremento de hora (S3):

- Presión única: decrementa los minutos en 1
- Presión sostenida: decremento rápido de minutos

Lógica de desbordamiento:

- Al llegar a 59 minutos y presionar S2: pasa a XX:00 (siguiente hora)
- Al llegar a 00 minutos y presionar S3: pasa a 59 minutos (hora anterior)

4.2.3. Análisis del Código - Parte 2

A continuación se desglosan las partes fundamentales del código implementado para el Reloj Digital:

1. Funciones de Actualización y Control de Tiempo

Estas funciones se encargan de traducir los valores de Hora y Minutos a los códigos de 7 segmentos correspondientes, separando decenas y unidades. Además, manejan la lógica de desbordamiento (de 59 a 00, y de 23 a 00).

```
1 void ActualizarDisplays(void){
2     datos[0] = codigosNum[Hora / 10]; // D1: decenas Hora
3     datos[1] = codigosNum[Hora % 10]; // D2: unidades Hora
4     datos[2] = codigosNum[Minutos / 10]; // D3: decenas Minutos
5     datos[3] = codigosNum[Minutos % 10]; // D4: unidades Minutos
6 }
7
8 void IncrementarTiempo(void){
9     Minutos++;
10    if(Minutos > 59){ Minutos = 0; Hora++; }
11    if(Hora > 23) Hora = 0; // Limite de 24 horas
12    ActualizarDisplays();
13 }
14
```

```

15 void DecrementarTiempo(void){
16     if(Minutos == 0){
17         Minutos = 59;
18         if(Hora == 0) Hora = 23;
19         else          Hora--;
20     } else {
21         Minutos--;
22     }
23     ActualizarDisplays();
24 }

```

Listing 5: Funciones lógicas del Reloj

2. Rutina de Servicio de Interrupción (ISR) Se configura para saltar cada 2.5 ms, garantizando una multiplexación imperceptible (10 ms el ciclo completo). También incrementa la variable `cntISR` que servirá como base de tiempo para contar los segundos en el *main*.

```

1 void __interrupt(high_priority) InterrupcionHP(void){
2     if(INTCONbits.TMR0IF){
3         // Recarga para 2.5 ms
4         TMR0H = 0xF1;
5         TMR0L = 0x5A;
6
7         // Apagar displays para evitar ghosting
8         LATCbits.LATC7 = 0; LATCbits.LATC6 = 0;
9         LATCbits.LATC1 = 0; LATCbits.LATC0 = 0;
10
11        // Multiplexacion
12        if(visDisplay){
13            LATD = (datos[multiplex] & 0xFE) | (LATDbits.LATD0);
14            switch(multiplex){
15                case 0: LATCbits.LATC7 = 1; break;
16                case 1: LATCbits.LATC6 = 1; break;
17                case 2: LATCbits.LATC1 = 1; break;
18                case 3: LATCbits.LATC0 = 1; break;
19            }
20        } else {
21            LATD = 0x00 | (LATDbits.LATD0);
22        }
23
24        multiplex++;
25        if(multiplex > 3) multiplex = 0;
26
27        // Tick: Base de tiempo para el main
28        cntISR++;
29
30        INTCONbits.TMR0IF = 0;
31    }
32 }

```

Listing 6: ISR para multiplexación (2.5 ms)

3. Bucle Principal y Lógica del Cronómetro/Botones El *main* evalúa constantemente los pulsantes con técnicas antirrebote y maneja el conteo de segundos usando los ticks generados por la interrupción.

```

1 while(1){
2     // ---- Tick de 500ms : (500/2.5 = 200 int.) ----
3     if(cntISR >= 200){
4         cntISR = 0;
5         MedioSecCount = !MedioSecCount;
6         LATDbits.LATD0 = !LATDbits.LATD0; // LED 1Hz
7
8         if(!is_set){

```

```

9         visDisplay = MedioSecCount; // Modo parpadeo inicial
10     } else {
11         visDisplay = 1;
12         // Cronometro normal
13         if(MedioSecCount == 0){
14             cuenta_Minutos++; // Cuenta de segundos
15             if(cuenta_Minutos >= 60){
16                 cuenta_Minutos = 0;
17                 IncrementarTiempo(); // Sube 1 minuto
18             }
19         }
20     }
21 }
22
23 // ---- Boton S2: incrementar ----
24 if(!PORTCbits.RC4){
25     btn_s2_cnt++;
26     if(btn_s2_cnt == 1){ // Presion unica
27         is_set = 1;
28         visDisplay = 1;
29         IncrementarTiempo();
30     }
31     // Presion sostenida rapida
32     if(btn_s2_cnt > 50 && (btn_s2_cnt % 10) == 0){
33         IncrementarTiempo();
34     }
35 } else {
36     btn_s2_cnt = 0;
37 }
38
39 // [Logica similar para S3 (RC5) y DecrementarTiempo()]
40 // ...
41
42 DelayXms(10); // Pausa no bloqueante
43 }

```

Listing 7: Bucle Principal y Botones (Fragmento)

Obtención de códigos: Los códigos se obtienen visualizando qué segmento se enciende para cada dígito, asignando un "1.^a dicho segmento y manteniendo el bit del punto decimal (dp) en "0". El código resultante en binario se convierte directamente a hexadecimal para ser enviado al Puerto D.

4.2.4. Diagrama de Flujo del Reloj

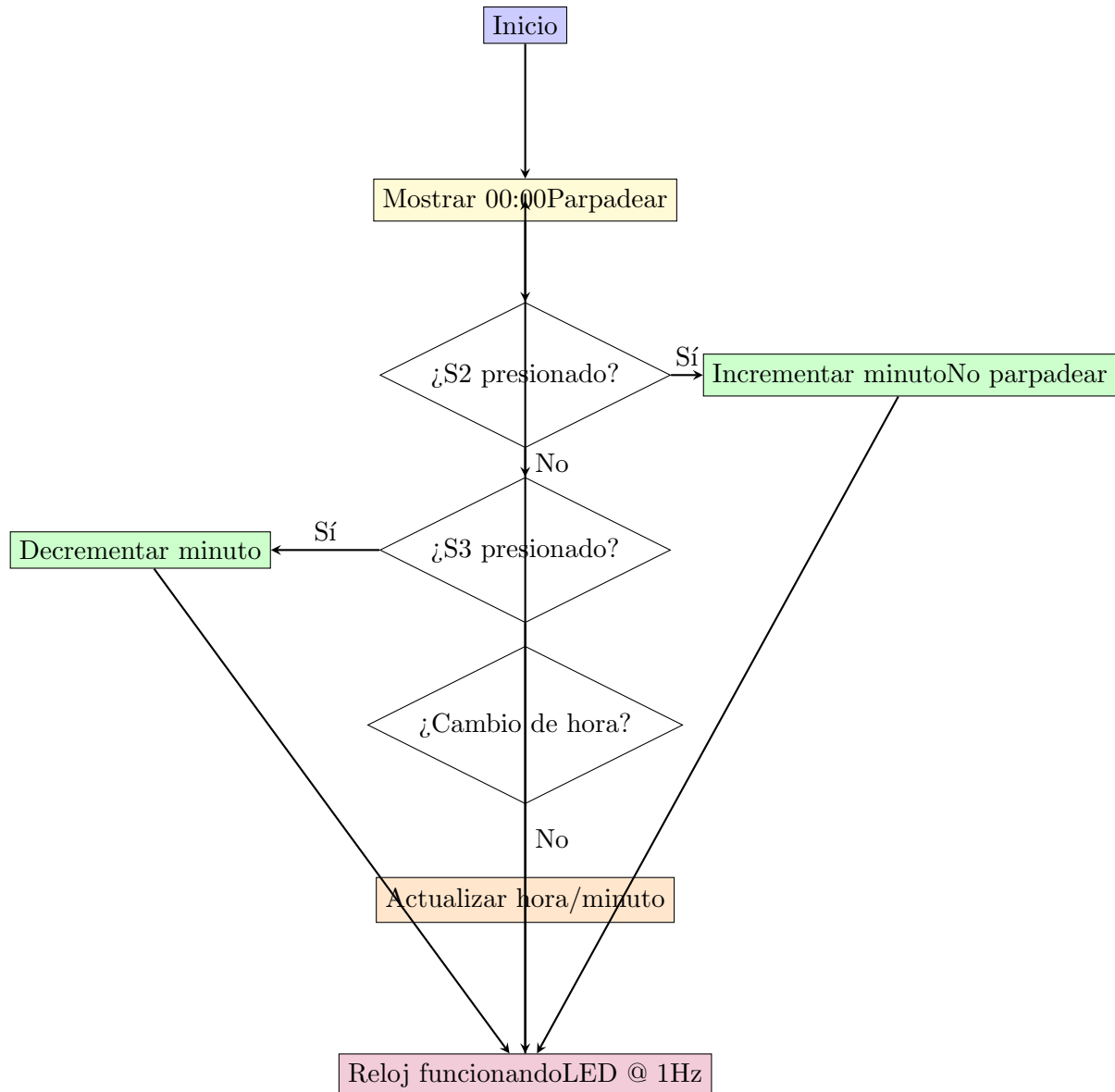


Figura 6: Flujo de funcionamiento del reloj digital

5. Pruebas y Verificaciones

5.1. Prueba de Reloj Digital

Procedimiento:

1. Observar inicialización en "00:00"
2. Presionar S2 múltiples veces
3. Presionar S3 para decrementar
4. Presionar S2 sostenido para incremento rápido

Resultados esperados:

- Inicialmente: "00:00" parpadeante indicando que no está configurado.

- Después de presionar S2: "00:01", el parpadeo se detiene.
- Incremento rápido al mantener presionado S2.
- Transición correcta de "00:59.^a "01:00".
- S3 decrementa correctamente y pasa de "00:00.^a "23:59".
- El LED RD0 parpadea constantemente a 1 Hz.

Resultado obtenido: Funcionamiento correcto

5.2. Parte 3: Funcionalidad Adicional (Mensaje Deslizante)

Agregar una funcionalidad propia de cada estudiante.

5.2.1. Descripción

Como funcionalidad propia, se implementó un letrero con efecto de desplazamiento continuo (Scroll) que muestra el nombre "JAIME CHIQUI". Debido a las limitaciones visuales de un display de 7 segmentos, algunas letras se adaptaron (por ejemplo, la "M" se visualiza como "nn" y la "Q" como un "9"). El sistema cuenta con controles de pausa y reanudación mediante los pulsantes.

5.2.2. Funcionamiento

- **Estado Inicial:** Al encender el sistema, los displays muestran la palabra "HoLA" de manera estática.
- **Iniciar / Reanudar (S2):** Al presionar S2, comienza el desplazamiento del texto hacia la izquierda. El texto avanza una posición cada 300 ms.
- **Pausar (S3):** Al presionar S3 mientras el texto se mueve, el desplazamiento se detiene, congelando las cuatro letras actuales en el display. Para continuar, se presiona nuevamente S3 o S2.

5.2.3. Cálculo de la Velocidad de Desplazamiento

Para lograr el efecto de movimiento del texto (Scroll) sin generar un parpadeo visual en los displays, el TMR0 nunca se altera y se mantiene fijo generando una interrupción cada **2.5 ms**.

En su lugar, la velocidad del texto se controla mediante un "temporizador por software" dentro del programa principal. Cada vez que ocurre la interrupción de 2.5 ms, se suma +1 a la variable `cntISR`.

Si el objetivo es que el texto avance una letra cada **300 ms** (una velocidad de lectura cómoda), el número de interrupciones necesarias se calcula de la siguiente manera:

$$\text{Ticks (interrupciones)} = \frac{\text{Tiempo de avance deseado}}{\text{Tiempo del TMR0}}$$

$$\text{Ticks} = \frac{300 \text{ ms}}{2,5 \text{ ms}} = 120 \text{ interrupciones}$$

Por lo tanto, en el código se define la variable `velocidad = 120`. Cuando el contador `cntISR` alcanza este valor, se actualiza la "ventana" de caracteres que observamos en los displays y el contador se reinicia.

5.2.4. Ilustración del Efecto Visual

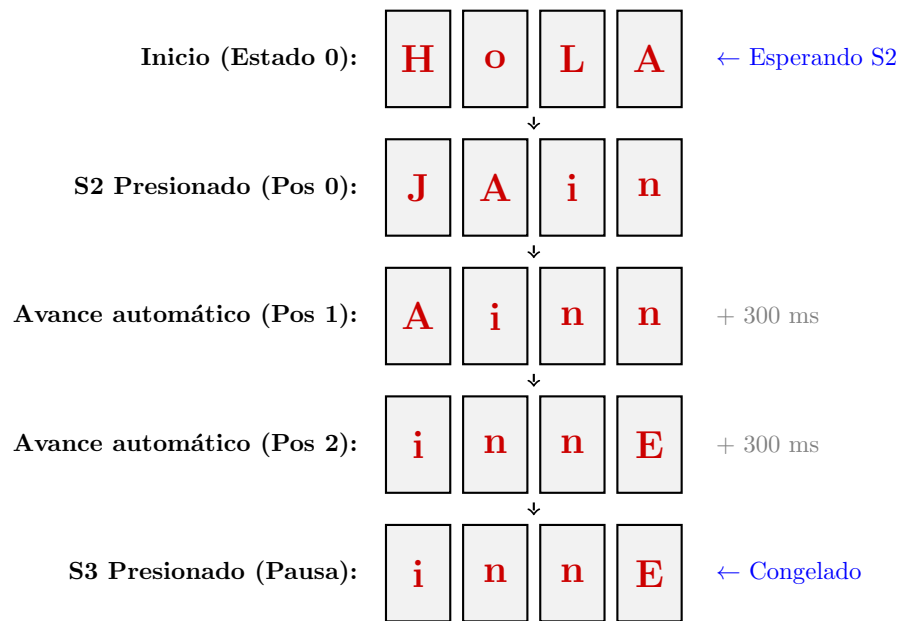


Figura 7: Representación gráfica del desplazamiento del texto y uso de los pulsantes

5.2.5. Análisis del Código - Parte 3

El código se estructuró para mantener la multiplexación estable a 2.5 ms por display, mientras que la lógica del deslizamiento se maneja mediante una máquina de estados en el ciclo principal.

1. Definición del Mensaje y Carga de Ventana

Se define un arreglo que contiene todo el mensaje codificado. La función `CargarVentana` toma un segmento de 4 caracteres del arreglo y lo asigna al buffer de los displays (`datos[]`).

```
1 // J A i n n E _ C H I Q U I _ _
2 #define TAM_MSG 15
3 const char mensaje[TAM_MSG] = {
4     0b01111000, // J
5     0b11101110, // A
6     0b01100000, // i
7     0b00101010, // n  <- usada para M
8     0b00101010, // n  <- usada para M
9     0b10011110, // E
10    0x00,        // _   espacio
11    0b10011100, // C
12    0b01101110, // H
13    0b01100000, // I   (1)
14    0b11100110, // Q   (9)
15    0b01111100, // U
16    0b01100000, // I   (1)
17    0x00,        // espacio final
18    0x00,        // espacio final
19 };
20
21 void CargarVentana(int pos){
22     int i;
23     for(i = 0; i < 4; i++){
24         if((pos + i) < TAM_MSG){
25             datos[i] = mensaje[pos + i];
26         } else {
```



```

27     datos[i] = 0x00; // Si se desborda, apaga el dígito
28 }
29 }
30 }

```

Listing 8: Lógica de la ventana deslizante

2. Máquina de Estados del Letrero (Main)

El avance del texto se rige por un contador alimentado por las interrupciones del Timer0. Cada 120 ticks ($120 \times 2.5 \text{ ms} = 300 \text{ ms}$), el texto avanza una posición.

```

1 // estadoScroll: 0=HoLA, 1=Corriendo, 2=Pausado
2 if(estadoScroll == 0){
3     // ESTADO 0: HoLA fijo (estado inicial)
4     visDisplay = 1;
5 } else if(estadoScroll == 1){
6     // ESTADO 1: Scroll corriendo
7     // 'velocidad' = 120 ticks (300 ms)
8     if(cntISR >= (unsigned int)velocidad){
9         cntISR = 0; // Resetear contador
10
11         CargarVentana(posScroll);
12         posScroll++; // Avanzar posición para la siguiente vez
13
14         // Al llegar al final del mensaje, reiniciar (loop)
15         if(posScroll >= TAM_MSG){
16             posScroll = 0;
17         }
18     }
19 } else if(estadoScroll == 2){
20     // ESTADO 2: Scroll en pausa
21     // El cuadro actual permanece congelado en los displays
22     cntISR = 0; // Evitar acumulación de tiempo
23 }

```

Listing 9: Máquina de estados en el Bucle Principal

6. Pruebas y Verificaciones - Parte 3

Para comprobar el correcto funcionamiento del letrero deslizante (Scroll) y la respuesta de los controles, se realizaron pruebas directas sobre el sistema evaluando el comportamiento del texto y de los botones S2 y S3.

6.1. Tabla de Pruebas Realizadas

En la Tabla 3 se resumen los ensayos efectuados y sus resultados prácticos.

Cuadro 3: Resultados de las pruebas del sistema de Scroll

Prueba	Acción realizada	Resultado observado	Estado
Inicialización	Encender o resetear el circuito.	El LED parpadea y los displays muestran "HoLA" de forma fija.	Correcto
Inicio Scroll	Presionar el botón S2.	El texto "JAIME CHIQUI" empieza a desplazarse a la izquierda.	Correcto
Pausa	Presionar el botón S3 en pleno movimiento.	El texto se congela inmediatamente en las letras actuales.	Correcto
Reanudación	Presionar S3 otra vez (o S2).	El texto continúa su marcha desde donde se quedó pausado.	Correcto
Bucle (Loop)	Esperar a que el texto termine.	Al finalizar el mensaje, vuelve a empezar automáticamente desde la 'J'.	Correcto

6.2. Análisis de Resultados

- **Velocidad del texto:** Se comprobó que el cambio de letra ocurre visualmente cada **300 ms**. Esto es exacto ya que el código espera a que ocurran 120 interrupciones del Timer0 ($120 \times 2,5 \text{ ms} = 300 \text{ ms}$), logrando una velocidad de lectura muy cómoda.
- **Control de botones:** Los botones responden correctamente y sin fallos por ruido mecánico. El retardo de 20 ms (antirebote) y el bucle de espera en el código garantizan que el estado cambie una sola vez por cada pulsación.
- **Adaptación de caracteres:** Al estar el texto en movimiento, las letras adaptadas como la 'M' (mostrada en dos dígitos como "nn") y la 'Q' (mostrada como un "9") se entienden y leen perfectamente en el display de 7 segmentos.

7. Códigos Completos y Repositorio

Los códigos completos para las tres partes de la práctica están disponibles en C/C++ implementados con MPLAB X IDE.

Archivos de código fuente:

- [Repositorio con todos los códigos de Práctica 3](#)

Videos de demostración:

- [Video Parte 1 - Multiplexación de displays "HoLA"](#)
- [Video Parte 2 - Reloj digital configurable](#)
- [Video Parte 3 - Implementación pantalla con Scroll](#)

Repositorio principal:

- [Repositorio Principal - Todas las Prácticas](#)

8. Preguntas

1. **¿Por qué es necesario obtener los códigos de encendido para cada número correspondientes al display de 7 segmentos?**

Los códigos de encendido son necesarios porque cada número o letra requiere un patrón específico de segmentos encendidos. Sin estos códigos, sería imposible mostrar la información correcta. Los códigos funcionan como una "tabla de traducción" entre lo que queremos mostrar en formato abstracto (un número) y qué pines físicos deben energizarse en el hardware.

Por ejemplo, en la distribución de bits utilizada en este proyecto [a b c d e f g dp], para mostrar el número "0" necesitamos encender los segmentos desde la a hasta la f y apagar el resto, lo que genera el código binario 0b11111100 (0xFC). Sin esta tabla indexada, el microcontrolador tendría que procesar ecuaciones lógicas complejas por cada segmento individual en tiempo de ejecución, reduciendo la eficiencia del programa.

2. **¿Cuál es la diferencia en los aspectos de programación si se hubiera utilizado un display de ánodo común con respecto a uno de cátodo común?**

Las diferencias principales radican en la polaridad de la señal lógica y la configuración de los datos:

Cátodo Común (usado en esta práctica):

- Los segmentos individuales se encienden con **lógica directa** (1 = encendido, 0 = apagado).
- El terminal común es el Cátodo y se conecta a Tierra (GND) a través del transistor de multiplexación.
- Ejemplo en la práctica (orden [a b c d e f g dp]): El número "0" corresponde a 0xFC=0b11111100.

Ánodo Común:

- Los segmentos se encienden con **lógica inversa** (0 = encendido, 1 = apagado).
- El terminal común es el Ánodo y se conecta permanentemente a +5V.
- Los códigos de la tabla serían exactamente el complemento binario (operación NOT) del cátodo común.
- Ejemplo para Ánodo Común: El número "0" correspondería a 0x03 (0b00000011).

En la programación, la estructura de multiplexación se mantiene idéntica, pero la tabla de constantes numéricas cambia por completo sus valores hexadecimales.



Figura 8: Display anodo y catodo comun

3. En el circuito de la figura 1.1: ¿cuál es la función de los 4 transistores ubicados en los terminales comunes de los displays? Al utilizarse displays de **Cátodo Común**, los 4 transistores actúan como **interruptores de control del lado bajo (Low-Side Switches)**. Sus funciones principales son:

- **Selección de dígito activo:** Cuando el microcontrolador satura la base de un transistor (enviando un 1 lógico a los pines de control del Puerto C), el transistor se comporta como un interruptor cerrado, conectando el cátodo común de ese display específico a Tierra (GND) y permitiendo que se ilumine.
- **Sincronización en la multiplexación:** Al apagar los 4 transistores antes de cambiar de dato, se evita el efecto de "ghosting" (rastros tenues de luz) en los displays adyacentes.
- **Amplificación de corriente:** Un solo pin del PIC18F4550 soporta un máximo de 25 mA. Si un display muestra el número "8", demanda corriente para 7 segmentos simultáneamente ($7 \times 15 \text{ mA} = 105 \text{ mA}$). El microcontrolador no puede suministrar esa corriente directamente sin quemarse; el transistor absorbe dicha corriente de carga hacia tierra, protegiendo al integrado.
- **Aislamiento:** Los transistores no conductores aseguran que su display correspondiente esté apagado sin interferir con otros.
- **Protección:** Los transistores aíslan el circuito de baja potencia (microcontrolador) del circuito de potencia (LEDs).

4. Presente una propuesta de circuito que permita encender un display de las mismas características del usado en esta práctica, con la diferencia de que se utilizará en un estadio, es decir con lámparas de mayor potencia: considere el caso de lámparas LED de 24V y 1A de consumo individual por segmento en cada display.

Para la adaptación del circuito multiplexado a un entorno de alta potencia (como un estadio con lámparas LED de 24V y un consumo de 1A por segmento), el microcontrolador PIC18F4550 no puede conectarse de forma directa debido a las limitaciones de corriente (máximo 25 mA por pin) y voltaje (5V max).

Por lo tanto, se diseñó una etapa de acoplamiento y amplificación utilizando transistores de estado sólido BJT de potencia. En la Figura 9 se presenta el esquema modificado en Proteus, donde se ejemplifica el desarrollo completo para un canal de segmento y la etapa de control para los dígitos multiplexados.

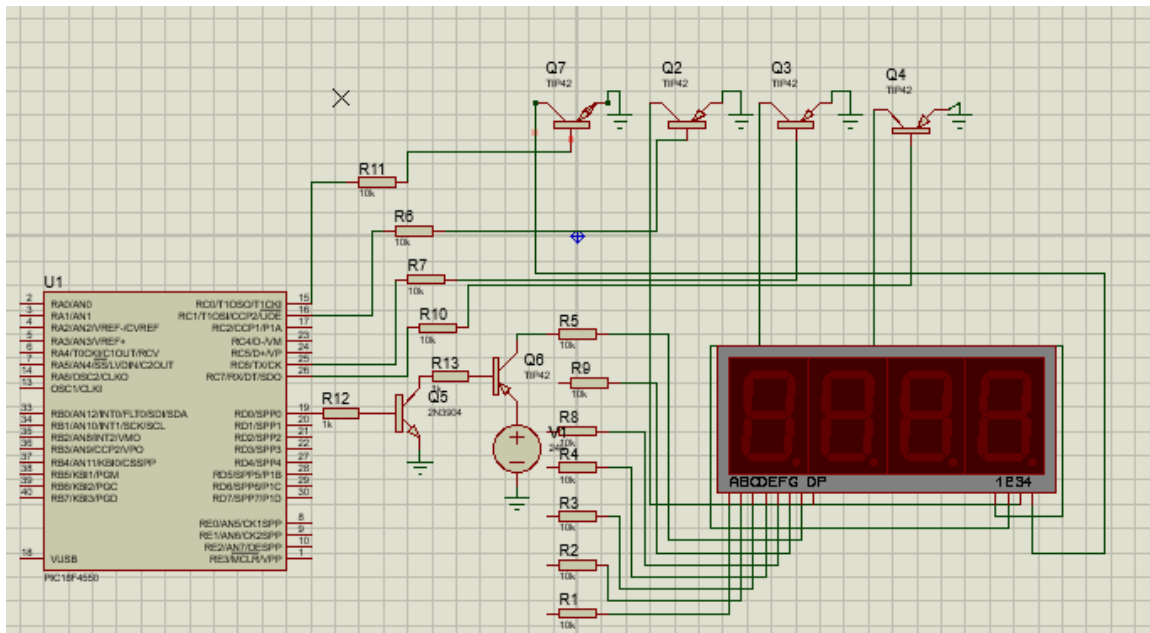


Figura 9: Esquema de la etapa de potencia propuesta. Se ilustra la configuración detallada para el segmento A y los pines comunes multiplexados.

Justificación técnica del diseño:

- **Control de Segmentos (Lado de Alto Voltaje):** Para conmutar los +24V de la fuente externa hacia los segmentos, se emplea un transistor PNP de potencia **TIP42** (capaz de manejar hasta 6A). Dado que el PIC entrega 5V, se añade un transistor NPN de baja señal **2N3904** como acoplador intermedio. Cuando el pin RD0 se pone en alto (5V), satura al 2N3904, lo que a su vez drena la corriente de la base del TIP42 hacia tierra, provocando su conducción y permitiendo el paso seguro de los 24V hacia el display. El resto de segmentos (B-G) se omiten en el diagrama por claridad, pero implementarían réplicas exactas de este bloque.
 - **Control de Dígitos Multiplexados (Retorno a Tierra):** Al encender un número denso (como el “8”), el pin común del dígito activo debe absorber de forma simultánea la corriente de los 7 segmentos ($7 \times 1 \text{ A} = 7 \text{ A}$). Para soportar esta carga crítica, se seleccionaron transistores NPN de potencia **TIP41** (o en su defecto Darlington TIP122) en lugar de transistores de baja señal, controlados eficientemente desde el puerto RC mediante resistencias de acoplamiento de 330Ω para garantizar la saturación completa.
5. Explique sus observaciones con respecto al cambio de lenguaje de programación desde ensamblador hacia C/C++.

Ventajas de C/C++:

- **Mayor legibilidad:** El código es mucho más fácil de entender y mantener. Una línea en C puede representar múltiples instrucciones en ensamblador.
- **Productividad:** Se escriben programas complejos mucho más rápidamente. No hay que preocuparse por cada ciclo de máquina.

- **Portabilidad:** El mismo código puede compilarse para diferentes microcontroladores con cambios mínimos.
- **Bibliotecas estándar:** Se dispone de librerías pre-escritas para operaciones comunes (`stdio.h`, `stdlib.h`, etc.).
- **Manejo de interrupciones:** En C es más simple usando decoradores como `_interrupt()`.
- **Depuración:** Herramientas de depuración más avanzadas disponibles.

Desventajas de C/C++:

- **Menos eficiencia:** El compilador no siempre genera el código más óptimo. En aplicaciones críticas, ensamblador es superior.
- **Mayor consumo de memoria:** Los programas en C generalmente usan más ROM y RAM que su equivalente en ensamblador optimizado.
- **Control limitado del hardware:** No se tiene acceso directo a cada ciclo de máquina.
- **Sobrecarga de ejecución:** Las funciones y abstracciones generan overhead adicional.

Conclusión personal: C/C++ es ideal para desarrollo rápido de prototipos y aplicaciones no críticas en tiempo. Ensamblador sigue siendo necesario para optimizaciones críticas y control de bajo nivel. La decisión depende de los requisitos del proyecto.

9. Conclusiones y Recomendaciones

9.1. Conclusiones

Con base a la resolución de esta practica se concluye que:

- La multiplexación mediante interrupciones del TMR0 cada 2.5 ms por dígito genera un ciclo de refresco de 100 Hz, superando el umbral mínimo de 50 Hz requerido para explotar la persistencia retiniana. Las pruebas físicas confirmaron que tiempos superiores a 10 ms por display introducen parpadeo perceptible, validando el rango de 2.5–5 ms como zona de operación óptima. De 10 ms en adelante hay un cambio notorio de parpadeo mientras mas se incrementa mas se aprecia este parpadeo.
- Los transistores NPN 2N3904 en configuración de interruptor de lado bajo son imprescindibles para drenar la corriente acumulada de hasta 7 segmentos simultáneos, protegiendo los pines del PIC18F4550 (máx. 25 mA). Además, apagarlos antes de cambiar el dato en el puerto eliminó eficazmente el efecto de *ghosting*.
- La separación de responsabilidades entre la ISR y el bucle principal resultó clave: la ISR se mantuvo mínima y determinista (recarga del TMR0, multiplexación y conteo de ticks), mientras que toda la lógica de alto nivel se delegó al *main*, produciendo un sistema predecible y libre de condiciones de carrera.
- El texto deslizante de la Parte 3 demostró que un único temporizador hardware puede servir múltiples propósitos lógicos. Acumular 120 ticks de 2.5 ms para generar un avance de 300 ms permitió controlar la velocidad del *scroll* con exactitud, sin alterar la configuración del TMR0.
- La programación en C con el compilador XC8 redujo la complejidad del desarrollo frente al ensamblador, permitiendo expresar algoritmos complejos de forma legible mediante arreglos en ROM, máquinas de estados y el atributo `__interrupt()`, aunque con un ligero incremento en el uso de memoria respecto a código ensamblador optimizado.

Referencias

- [1] Microchip Technology Inc., *PIC18F2455/2550/4455/4550 Data Sheet: High Performance, Enhanced Flash, USB Microcontrollers with nanoWatt Technology*, DS39632E, 2024. [En línea]. Disponible en: <https://ww1.microchip.com/downloads/en/devicedoc/39632e.pdf>
- [2] Nexperia, *7-Segment LED Display Decoder Driver Application Notes*, 2023.
- [3] Texas Instruments, *LED Display Multiplexing Techniques*, Technical Report, 2023.

10. Anexos

10.0.1. Código completo parte 1

El programa en lenguaje C para la parte 1 es:

```
1 /*
2  * File:   Codigo_Parte1.c
3  * Author: JAIME CHIQUI
4  *
5  * Created on May 26, 2026, 3:56 PM
6  */
7 // PRACTICA 3 - PARTE 1
8 // -- HoLA cor TMR0 e interrupciones
9 // PIC18F4550 Configuration Bit Settings
10
11 // 'C' source line config statements
12
13 // CONFIG1L
14 #pragma config PLLDIV = 5          // PLL Prescaler Selection bits (No prescale (4
    MHz oscillator input drives PLL directly))
15 #pragma config CPUDIV = OSC1_PLL2 // System Clock Postscaler Selection bits ([
    Primary Oscillator Src: /1][96 MHz PLL Src: /2])
16 #pragma config USBDIV = 1          // USB Clock Selection bit (used in Full-Speed
    USB mode only; UCFG:FSEN = 1) (USB clock source comes directly from the
    primary oscillator block with no postscale)
17
18 // CONFIG1H
19 #pragma config FOSC = HSPLL_HS     // Oscillator Selection bits (HS oscillator, PLL
    enabled (HSPLL))
20 #pragma config FCMEN = OFF          // Fail-Safe Clock Monitor Enable bit (Fail-Safe
    Clock Monitor disabled)
21 #pragma config IESO = OFF           // Internal/External Oscillator Switchover bit (
    Oscillator Switchover mode disabled)
22
23 // CONFIG2L
24 #pragma config PWRT = ON            // Power-up Timer Enable bit (PWRT enabled)
25 #pragma config BOR = ON            // Brown-out Reset Enable bits (Brown-out Reset
    enabled in hardware only (SBOREN is disabled))
26 #pragma config BORV = 3            // Brown-out Reset Voltage bits (Minimum setting
    2.05V)
27 #pragma config VREGEN = OFF         // USB Voltage Regulator Enable bit (USB voltage
    regulator disabled)
28
29 // CONFIG2H
30 #pragma config WDT = OFF            // Watchdog Timer Enable bit (WDT disabled (
    control is placed on the SWDTEN bit))
31 #pragma config WDTPS = 32768       // Watchdog Timer Postscale Select bits
    (1:32768)
32
33 // CONFIG3H
34 #pragma config CCP2MX = ON          // CCP2 MUX bit (CCP2 input/output is
    multiplexed with RC1)
35 #pragma config PBADEN = OFF         // PORTB A/D Enable bit (PORTB<4:0> pins are
    configured as digital I/O on Reset)
36 #pragma config LPT1OSC = OFF        // Low-Power Timer 1 Oscillator Enable bit (
    Timer1 configured for higher power operation)
37 #pragma config MCLRE = ON           // MCLR Pin Enable bit (MCLR pin enabled; RE3
    input pin disabled)
38
39 // CONFIG4L
40 #pragma config STVREN = ON          // Stack Full/Underflow Reset Enable bit (Stack
    full/underflow will cause Reset)
41 #pragma config LVP = OFF            // Single-Supply ICSP Enable bit (Single-Supply
```



```

        ICSP enabled)
42 #pragma config ICPRT = OFF          // Dedicated In-Circuit Debug/Programming Port (
        ICPRT) Enable bit (ICPRT disabled)
43 #pragma config XINST = OFF          // Extended Instruction Set Enable bit (
        Instruction set extension and Indexed Addressing mode disabled (Legacy mode)
        )
44 // ** Apagar LVP para no tener corriente parasita *****
45
46 // CONFIG5L
47 #pragma config CP0 = OFF             // Code Protection bit (Block 0 (000800-001FFFh)
        is not code-protected)
48 #pragma config CP1 = OFF             // Code Protection bit (Block 1 (002000-003FFFh)
        is not code-protected)
49 #pragma config CP2 = OFF             // Code Protection bit (Block 2 (004000-005FFFh)
        is not code-protected)
50 #pragma config CP3 = OFF             // Code Protection bit (Block 3 (006000-007FFFh)
        is not code-protected)
51
52 // CONFIG5H
53 #pragma config CPB = OFF             // Boot Block Code Protection bit (Boot block
        (000000-0007FFFh) is not code-protected)
54 #pragma config CPD = OFF             // Data EEPROM Code Protection bit (Data EEPROM
        is not code-protected)
55
56 // CONFIG6L
57 #pragma config WRT0 = OFF            // Write Protection bit (Block 0 (000800-001FFFh)
        ) is not write-protected)
58 #pragma config WRT1 = OFF            // Write Protection bit (Block 1 (002000-003FFFh)
        ) is not write-protected)
59 #pragma config WRT2 = OFF            // Write Protection bit (Block 2 (004000-005FFFh)
        ) is not write-protected)
60 #pragma config WRT3 = OFF            // Write Protection bit (Block 3 (006000-007FFFh)
        ) is not write-protected)
61
62 // CONFIG6H
63 #pragma config WRTC = OFF            // Configuration Register Write Protection bit (
        Configuration registers (300000-3000FFFh) are not write-protected)
64 #pragma config WRTB = OFF            // Boot Block Write Protection bit (Boot block
        (000000-0007FFFh) is not write-protected)
65 #pragma config WRTD = OFF            // Data EEPROM Write Protection bit (Data EEPROM
        is not write-protected)
66
67 // CONFIG7L
68 #pragma config EBTR0 = OFF           // Table Read Protection bit (Block 0
        (000800-001FFFh) is not protected from table reads executed in other blocks)
69 #pragma config EBTR1 = OFF           // Table Read Protection bit (Block 1
        (002000-003FFFh) is not protected from table reads executed in other blocks)
70 #pragma config EBTR2 = OFF           // Table Read Protection bit (Block 2
        (004000-005FFFh) is not protected from table reads executed in other blocks)
71 #pragma config EBTR3 = OFF           // Table Read Protection bit (Block 3
        (006000-007FFFh) is not protected from table reads executed in other blocks)
72
73 // CONFIG7H
74 #pragma config EBTRB = OFF           // Boot Block Table Read Protection bit (Boot
        block (000000-0007FFFh) is not protected from table reads executed in other
        blocks)
75
76 // #pragma config statements should precede project file includes.
77 // Use project enums instead of #define for ON and OFF.
78
79 #include <xc.h>
80 // =====
81 //
        VARIABLES GLOBALES

```

```

82 // =====
83 volatile char datos[4]      = {0x00, 0x00, 0x00, 0x00}; // 4 DISPLAYS OFF
84 volatile char multiplex     = 0; // digito activo 0-3
85 volatile unsigned char visDisplay = 1; // 1=mostrar 0=apagar displays
86 volatile char estado = 0; // 0 = Fijo (S3), 1 = Parpadeo (S2)
87
88 // Matriz de n meros 0-9
89 // Orden de bits: [a b c d e f g dp] - CAT COMUN
90 const char codigosNum[10] = {
91     0b11111100, // 0
92     0b01100000, // 1
93     0b11011010, // 2
94     0b11110010, // 3
95     0b01100110, // 4
96     0b10110110, // 5
97     0b10111110, // 6
98     0b11100000, // 7
99     0b11111110, // 8
100    0b11110110 // 9
101 };
102 // Matriz de letras (A, b, C, d, E, F, G, H, h, i, J, L, n, o, P, q, r, S, t, U,
    u, y)
103 const char codigosLetras[] = {
104     0b11101110, // A
105     0b00111110, // b
106     0b10011100, // C
107     0b01111010, // d
108     0b10011110, // E
109     0b10001110, // F
110     0b10111110, // G
111     0b01101110, // H  <- ind 7
112     0b00101110, // h
113     0b00100000, // i
114     0b01111000, // J
115     0b00011100, // L  <- ind 11
116     0b00101010, // n
117     0b00111010, // o  <- ind 13
118     0b11001110, // P
119     0b11110110, // q
120     0b00001010, // r
121     0b10110110, // S
122     0b00011110, // t
123     0b01111100, // U
124     0b00111000, // u
125     0b01110110 // y
126 };
127
128 // =====
129 //                      DELAYS
130 // =====
131 void Delay1ms(){
132     int veces;
133     for ( veces=0; veces <= 860; veces++ );
134 }
135
136 void DelayXms(int Milis){
137     for(int i=0; i<= Milis; i++){
138         Delay1ms();
139     }
140 }
141 void MostrarSecuencia(void){
142     int d, n;
143     // Recorre los 4 displays: 0=D1, 1=D2, 2=D3, 3=D4

```

```

144     for(d = 0; d < 4; d++){
145         for(n = 0; n <= 9; n++){
146             // Pone el numero en el display activo, el resto apagado
147             datos[0] = 0x00;
148             datos[1] = 0x00;
149             datos[2] = 0x00;
150             datos[3] = 0x00;
151             datos[d] = codigosNum[n];    // solo el display d muestra n
152             DelayXms(250);                // 500ms por numero
153         }
154         // Display se apaga al terminar (datos[d] queda en 9, luego se limpia)
155         datos[d] = 0x00;
156         DelayXms(100);                    // pausa apagado entre displays
157     }
158 }
159
160 void __interrupt(high_priority) InterrupcionHP(void){
161     // =====
162     // ISR ALTA PRIORIDAD - TMRO 16Bits
163     // =====
164     if(INTCONbits.TMROIF){
165         if(estado == 0){
166             // MOD0 S3: Carga 2.5ms (HoLA se ve s lido y fijo sin parpadear)
167             TMROH = 0xF1; TMROL = 0x5A;
168         } else{
169             // -----
170             //             2.5 [ms]
171             // -----
172             TMROH = 0xF1; TMROL = 0x5A;
173             // -----
174             //             5 [ms]
175             // -----
176             TMROH = 0xE2; TMROL = 0xB4;
177             // -----
178             //             10 [ms]
179             // -----
180             TMROH = 0xC5; TMROL = 0x68;
181             // =====
182             //             25[ms]
183             // =====
184             TMROH = 0x6D; TMROL = 0x84;
185             // =====
186             //             50 [ms]
187             // =====
188             TMROH = 0xB6; TMROL = 0x82;
189         }
190         // Evita superposici n
191         LATCbits.LATC7 = 0;
192         LATCbits.LATC6 = 0;
193         LATCbits.LATC1 = 0;
194         LATCbits.LATC0 = 0;
195         // Multiplexacion
196         if(visDisplay){
197             LATD = (datos[multiplex] & 0xFE) | (LATDbits.LATD0);
198             // 0xFE = 11111110 -> limpia bit0 (dp) pero conserva RDO del LED
199
200             switch(multiplex){
201                 case 0: LATCbits.LATC7 = 1; break; // D1
202                 case 1: LATCbits.LATC6 = 1; break; // D2
203                 case 2: LATCbits.LATC1 = 1; break; // D3
204                 case 3: LATCbits.LATC0 = 1; break; // D4
205             }
206         } else {

```

```

207     LATD = 0x00 | (LATDbits.LATD0);
208 }
209     multiplex++;
210     if(multiplex > 3) multiplex = 0;
211
212     INTCONbits.TMR0IF = 0;
213     return;
214 }
215 }
216
217 // =====
218 //             MAIN
219 // =====
220 void main(void){
221     // --- CONFIGURACION CRITICA PARA USAR RC4 y RC5 ---
222     UCONbits.USBEN = 0;    // Deshabilita el modulo USB
223     UCFGbits.UTRDIS = 1;  // Convierte a E/S digital
224     // -----
225
226     // Configuracion de entrada / salida
227     TRISD = 0;             // Puerto D todo salida
228     TRISCbits.TRISC0 = 0;  // RC0=D4 salida
229     TRISCbits.TRISC1 = 0;  // RC1=D3 salida
230     TRISCbits.TRISC6 = 0;  // RC6=D2 salida
231     TRISCbits.TRISC7 = 0;  // RC7=D1 salida
232     TRISC |= 0b00110000;   // RC4=S2 y RC5=S3 entradas(en 1)
233
234     // OFF Display (evita ghosting: int. repentina)
235     LATD = 0;
236     LATCbits.LATC7 = 0;    // D1
237     LATCbits.LATC6 = 0;    // D2
238     LATCbits.LATC1 = 0;    // D3
239     LATCbits.LATC0 = 0;    // D4
240
241     if(estado == 0){
242         // MODO S3: Carga 2.5ms (HoLA se ve s lido y fijo sin parpadear)
243         TMR0H = 0xF1; TMR0L = 0x5A;
244     } else{
245         //=====
246         // Recargar TMR0 segun el tiempo elegido
247         //=====
248         // Fosc=48MHz Tcy=1/12MHz=83.33ns
249         // casos 1,2,3,4 -> Prescaler 1:8 Tpre=666.67ns
250         // caso 5        -> Prescaler 1:32 Tpre=2666.67ns
251         // Carga = 65536 - (Tiempo / Tpre)
252
253         // -----
254         //             2.5 [ms]
255         // -----
256         // Conteo = 2.5ms / 666.67ns = 3750
257         // Carga = 65536 - 3750 = 61786 = 0xF15A
258         // TMR0H = 0xF1; TMR0L = 0x5A;
259
260         //-----
261         //             5 [ms]
262         // -----
263         // Instrucciones = 5ms / 666.67ns = 7500
264         // Carga = 65536 - 7500 = 58036 = 0xE2B4
265         //TMR0H = 0xE2; TMR0L = 0xB4;
266
267         // -----
268         //             10 [ms]
269         // -----

```

```

270      // Conteo = 10ms / 666.67ns = 15000
271      // Carga = 65536 - 15000 = 50536 = 0xC568
272      //TMR0H = 0xC5; TMR0L = 0x68;
273
274      //=====
275      //          25[ms]
276      //=====
277      // Conteo = 25ms / 666.67ns = 37500
278      // Carga = 65536 - 37500 = 28036 = 0x6D84
279      // TMR0H = 0x6D; TMR0L = 0x84;
280      //=====
281      //          50 [ms]
282      //=====
283      // Conteo = 50ms / 2666.67ns = 18750
284      // Carga = 65536 - 18750 = 46786 = 0xB682
285      TMR0H = 0xB6; TMR0L = 0x82;
286  }
287
288  // =====
289  // INICIALIZACION: RDO -> 10 pulsos de 50ms, luego ON
290  // =====
291  for(int i=0; i<10; i++){
292      LATDbits.LATD0 = !LATDbits.LATD0; // toggle
293      DelayXms(50);
294  }
295  LATDbits.LATD0 = 1; // ON Led
296
297  // =====
298  // CARGA LA PALABRA HoLA EN EL ARREGLO DATOS
299  // codigosLetras[7]=H [13]=o [11]=L [0]=A
300  // datos[0]=D1=H datos[1]=D2=o datos[2]=D3=L datos[3]=D4=A
301  // =====
302  datos[0] = codigosLetras[7]; // H
303  datos[1] = codigosLetras[13]; // o
304  datos[2] = codigosLetras[11]; // L
305  datos[3] = codigosLetras[0]; // A
306
307  // =====
308  // SECUENCIA 0-9
309  // =====
310  // TOCONbits.TMR0ON = 1; // Encender TMR0 primero
311  // MostrarSecuencia();
312
313  // =====
314  // Configuracion de TMR0 - MODO 16 BITS
315  //
316  // TOCON:
317  // bit7: TMR0ON = 0 (apagado por ahora)
318  // bit6: T08BIT = 0 (modo 16-bit) <- 0 en 16bit, 1 en 8bit
319  // bit5: T0CS = 0 (reloj interno Fosc/4)
320  // bit4: T0SE = 0 (no importa en interno)
321  // bit3: PSA = 0 (prescaler activo)
322  // bit2-0: TOPS prescaler:
323  // 000=1:2 001=1:4 010=1:8 011=1:16
324  // 100=1:32 101=1:64 110=1:128 111=1:256
325  // =====
326  // -----
327  // Casos 1, 2, 3, 4 -> Prescaler 1:8 TOPS=010
328  // -----
329  // TOCON = 0b00000010;
330
331  // -----
332  // caso 5 -> Prescaler 1:32 TOPS=100

```

```

333 // -----
334 TOCON = 0b000000100;
335
336 // Configuración de interrupciones
337 RCONbits.IPEN = 1; // Prioridades habilitadas
338 INTCON2bits.TMROIP = 1; // TMRO alta prioridad
339 INTCONbits.TMROIE = 1; // Interrupción TMRO ON
340 INTCONbits.TMROIF = 0; // Limpiar bandera
341 INTCONbits.GIEH = 1; // Global alta ON
342 INTCONbits.GIEL = 1; // Global baja ON
343
344 TOCONbits.TMROON = 1; // Encender TMRO
345 while(1){
346     // MODO S2: TEST DE VELOCIDAD LENTO
347     if(PORTCbits.RC4 == 0){
348         DelayXms(20);
349         if(PORTCbits.RC4 == 0){
350             estado = 1; // La interrupción empieza a usar el
Test lento
351             while(PORTCbits.RC4 == 0);
352         }
353     }
354
355     // MODO S3: FIJO Y PERFECTO
356     if(PORTCbits.RC5 == 0){
357         DelayXms(20);
358         if(PORTCbits.RC5 == 0){
359             estado = 0; // La interrupción regresa a 2.5ms
360             while(PORTCbits.RC5 == 0);
361         }
362     }
363 }
364 }

```

Listing 10: Estructura general del programa - Práctica 3 - Parte 1

10.0.2. Código completo parte 2

El programa en lenguaje C para la parte 2 es:

```

1 /*
2  * File:    Pr3_Codigo_Parte2.c
3  * Author:  JAIME CHIQUI
4  *
5  * Created on 28 de mayo de 2026, 10:42
6  */
7 // PRACTICA 3 - PARTE 2: RELOJ HH:MM
8 // PIC18F4550 Configuration Bit Settings
9
10 // 'C' source line config statements
11
12 // CONFIG1L
13 #pragma config PLLDIV = 5 // PLL Prescaler Selection bits (No prescale (4
    MHz oscillator input drives PLL directly))
14 #pragma config CPUDIV = OSC1_PLL2 // System Clock Postscaler Selection bits ([
    Primary Oscillator Src: /1][96 MHz PLL Src: /2])
15 #pragma config USBDIV = 1 // USB Clock Selection bit (used in Full-Speed
    USB mode only; UCFG:FSEN = 1) (USB clock source comes directly from the
    primary oscillator block with no postscale)
16
17 // CONFIG1H
18 #pragma config FOSC = HSPLL_HS // Oscillator Selection bits (HS oscillator, PLL
    enabled (HSPLL))

```

```

19 #pragma config FCMEN = OFF      // Fail-Safe Clock Monitor Enable bit (Fail-Safe
    Clock Monitor disabled)
20 #pragma config IESO = OFF      // Internal/External Oscillator Switchover bit (
    Oscillator Switchover mode disabled)
21
22 // CONFIG2L
23 #pragma config PWRT = ON       // Power-up Timer Enable bit (PWRT enabled)
24 #pragma config BOR = ON       // Brown-out Reset Enable bits (Brown-out Reset
    enabled in hardware only (SBOREN is disabled))
25 #pragma config BORV = 3       // Brown-out Reset Voltage bits (Minimum setting
    2.05V)
26 #pragma config VREGEN = OFF    // USB Voltage Regulator Enable bit (USB voltage
    regulator disabled)
27
28 // CONFIG2H
29 #pragma config WDT = OFF       // Watchdog Timer Enable bit (WDT disabled (
    control is placed on the SWDTEN bit))
30 #pragma config WDTPS = 32768  // Watchdog Timer Postscale Select bits
    (1:32768)
31
32 // CONFIG3H
33 #pragma config CCP2MX = ON     // CCP2 MUX bit (CCP2 input/output is
    multiplexed with RC1)
34 #pragma config PBADEN = OFF    // PORTB A/D Enable bit (PORTB<4:0> pins are
    configured as digital I/O on Reset)
35 #pragma config LPT1OSC = OFF   // Low-Power Timer 1 Oscillator Enable bit (
    Timer1 configured for higher power operation)
36 #pragma config MCLRE = ON      // MCLR Pin Enable bit (MCLR pin enabled; RE3
    input pin disabled)
37
38 // CONFIG4L
39 #pragma config STVREN = ON     // Stack Full/Underflow Reset Enable bit (Stack
    full/underflow will cause Reset)
40 #pragma config LVP = OFF       // Single-Supply ICSP Enable bit (Single-Supply
    ICSP enabled)
41 #pragma config ICPRT = OFF     // Dedicated In-Circuit Debug/Programming Port (
    ICPRT) Enable bit (ICPRT disabled)
42 #pragma config XINST = OFF     // Extended Instruction Set Enable bit (
    Instruction set extension and Indexed Addressing mode disabled (Legacy mode)
    )
43 // ** Apagar LVP para no tener corriente parasita *****
44
45 // CONFIG5L
46 #pragma config CP0 = OFF       // Code Protection bit (Block 0 (000800-001FFFh)
    is not code-protected)
47 #pragma config CP1 = OFF       // Code Protection bit (Block 1 (002000-003FFFh)
    is not code-protected)
48 #pragma config CP2 = OFF       // Code Protection bit (Block 2 (004000-005FFFh)
    is not code-protected)
49 #pragma config CP3 = OFF       // Code Protection bit (Block 3 (006000-007FFFh)
    is not code-protected)
50
51 // CONFIG5H
52 #pragma config CPB = OFF       // Boot Block Code Protection bit (Boot block
    (000000-0007FFFh) is not code-protected)
53 #pragma config CPD = OFF       // Data EEPROM Code Protection bit (Data EEPROM
    is not code-protected)
54
55 // CONFIG6L
56 #pragma config WRT0 = OFF      // Write Protection bit (Block 0 (000800-001FFFh)
    ) is not write-protected)
57 #pragma config WRT1 = OFF      // Write Protection bit (Block 1 (002000-003FFFh)
    ) is not write-protected)

```

```

58 #pragma config WRT2 = OFF          // Write Protection bit (Block 2 (004000-005FFFh
   ) is not write-protected)
59 #pragma config WRT3 = OFF          // Write Protection bit (Block 3 (006000-007FFFh
   ) is not write-protected)
60
61 // CONFIG6H
62 #pragma config WRTC = OFF          // Configuration Register Write Protection bit (
   Configuration registers (300000-3000FFh) are not write-protected)
63 #pragma config WRTB = OFF          // Boot Block Write Protection bit (Boot block
   (000000-0007FFh) is not write-protected)
64 #pragma config WRTD = OFF          // Data EEPROM Write Protection bit (Data EEPROM
   is not write-protected)
65
66 // CONFIG7L
67 #pragma config EBTR0 = OFF          // Table Read Protection bit (Block 0
   (000800-001FFFh) is not protected from table reads executed in other blocks)
68 #pragma config EBTR1 = OFF          // Table Read Protection bit (Block 1
   (002000-003FFFh) is not protected from table reads executed in other blocks)
69 #pragma config EBTR2 = OFF          // Table Read Protection bit (Block 2
   (004000-005FFFh) is not protected from table reads executed in other blocks)
70 #pragma config EBTR3 = OFF          // Table Read Protection bit (Block 3
   (006000-007FFFh) is not protected from table reads executed in other blocks)
71
72 // CONFIG7H
73 #pragma config EBTRB = OFF          // Boot Block Table Read Protection bit (Boot
   block (000000-0007FFh) is not protected from table reads executed in other
   blocks)
74
75 // #pragma config statements should precede project file includes.
76 // Use project enums instead of #define for ON and OFF.
77 #include <xc.h>
78
79 // =====
80 //                               VARIABLES GLOBALES
81 // =====
82 volatile char datos[4]              = {0x00, 0x00, 0x00, 0x00};
83 volatile char multiplex              = 0;
84 volatile unsigned char visDisplay = 1; // 1=mostrar 0=apagar
85
86 // Flags y contadores de ISR
87 volatile unsigned int cntISR         = 0; // cuenta interrupciones para 1 s
88 int Hora = 0;
89 int Minutos = 0;
90 char is_set = 0;
91
92 //Codigo de los numeros 0-9 (c todo com n, orden [a b c d e f g dp])
93 const char codigosNum[10] = {
94     0b11111100, // 0
95     0b01100000, // 1
96     0b11011010, // 2
97     0b11110010, // 3
98     0b01100110, // 4
99     0b10110110, // 5
100    0b10111110, // 6
101    0b11100000, // 7
102    0b11111110, // 8
103    0b11110110  // 9
104 };
105
106 // =====
107 //                               DELAYS
108 // =====
109 void Delay1ms(){

```



```

110     int veces;
111     for ( veces=0; veces <= 860; veces++ );
112 }
113
114 void DelayXms(int Milis){
115     for(int i=0; i<= Milis; i++){
116         Delay1ms();
117     }
118 }
119
120 // =====
121 //          ACTUALIZAR DISPLAYS CON MM:SS
122 // =====
123 void ActualizarDisplays(void){
124     datos[0] = codigosNum[Hora / 10]; // D1: decenas Hora
125     datos[1] = codigosNum[Hora % 10]; // D2: unidades Hora
126     datos[2] = codigosNum[Minutos / 10]; // D3: decenas Minutos
127     datos[3] = codigosNum[Minutos % 10]; // D4: unidades Minutos
128
129     // ejemplo: 37 / 10 = 3.7 (int en c: 3)
130     //           37 % 10 = 7
131 }
132 void IncrementarTiempo(void){
133     Minutos++;
134     if(Minutos > 59){ Minutos = 0; Hora++; }
135     if(Hora > 23) Hora = 0; // Horas
136     ActualizarDisplays();
137 }
138
139 void DecrementarTiempo(void){
140     if(Minutos == 0){
141         Minutos = 59;
142         if(Hora == 0) Hora = 23; // hras
143         else Hora--;
144     } else {
145         Minutos--;
146     }
147     ActualizarDisplays();
148 }
149 // =====
150 //          ISR ALTA PRIORIDAD - TMRO 16 bits
151 // =====
152 void __interrupt(high_priority) InterrupcionHP(void){
153     if(INTCONbits.TMROIF){
154         // Para 2.5 ms debes cargar:
155         TMROH = 0xF1; // Parte alta
156         TMROL = 0x5A; // Parte baja
157         // Apagar todos los digitos antes de cambiar (evita ghosting)
158         LATCbits.LATC7 = 0; // D1
159         LATCbits.LATC6 = 0; // D2
160         LATCbits.LATC1 = 0; // D3
161         LATCbits.LATC0 = 0; // D4
162
163         // Multiplexaci n
164         if(visDisplay){
165             LATD = (datos[multiplex] & 0xFE) | (LATDbits.LATD0);
166             switch(multiplex){
167                 case 0: LATCbits.LATC7 = 1; break;
168                 case 1: LATCbits.LATC6 = 1; break;
169                 case 2: LATCbits.LATC1 = 1; break;
170                 case 3: LATCbits.LATC0 = 1; break;
171             }
172         } else {

```

```

173     LATD = 0x00 | (LATDbits.LATD0);
174 }
175
176 // multiplex incre. siempre
177 multiplex++;
178 if(multiplex > 3) multiplex = 0;
179
180 // Tick: 200 interrupciones = 500ms / 5ms
181 cntISR++; //
182
183 INTCONbits.TMROIF = 0;
184 }
185 }
186
187 // =====
188 //                               MAIN
189 // =====
190 void main(void){
191     unsigned char MedioSecCount = 0; // 0 o 1, alterna cada 500ms: 0.5seg
192     unsigned char cuenta_Minutos = 0;
193
194     int btn_s2_cnt = 0;
195     int btn_s3_cnt = 0;
196
197     // --- CONFIGURACION CRITICA PARA USAR RC4 y RC5 ---
198     UCONbits.USBEN = 0; // Deshabilita el modulo USB
199     UCFGbits.UTRDIS = 1; // Deshabilita el transceptor USB (Convierte a E/S
digital)
200     // -----
201
202     TRISD = 0;
203     TRISCbits.TRISC0 = 0;
204     TRISCbits.TRISC1 = 0;
205     TRISCbits.TRISC6 = 0;
206     TRISCbits.TRISC7 = 0;
207     TRISC |= 0b00110000; // Activar s2 y s3
208
209     LATD = 0;
210     LATCbits.LATC7 = 0;
211     LATCbits.LATC6 = 0;
212     LATCbits.LATC1 = 0;
213     LATCbits.LATC0 = 0;
214     // == INTERRUPTACIONES DE 2.5 ms
215     // Tiempo total del ciclo = 4 displays * 2.5 ms = 10 ms
216     // =====
217     // TMR0: 5ms (Fosc=48MHz, 16-bit, Prescaler 1:8)
218     // =====
219     // Tcy = 1 / (48MHz / 4) = 83.33ns
220     // Tick = Tcy * 8 (Prescaler) = 666.67ns
221     // Conteos = 2.5ms / 666.67ns = 3750
222     // Carga = 65536 - 3750 = 61786 = 0xF15A
223     TMROH = 0xF1;
224     TMROL = 0x5A;
225     TOCON = 0b00000010;
226
227     // Configuración del TOCON:
228     // bit 7: TMR0ON = 0 (Apagado por ahora)
229     // bit 6: T08BIT = 0 (Configurado en 16 bits)
230     // bit 5: T0CS = 0 (Reloj interno Fosc/4)
231     // bit 4: T0SE = 0 (No reloj interno)
232     // bit 3: PSA = 0 (Prescaler activado)
233     // bit 2-0: T0PS = 010 (Prescaler 1:8)
234

```

```

235 RCONbits.IPEN      = 1;
236 INTCON2bits.TMROIP = 1;
237 INTCONbits.TMROIE  = 1;
238 INTCONbits.TMROIF  = 0;
239 INTCONbits.GIEH    = 1;
240 INTCONbits.GIEL    = 1;
241 TOCONbits.TMROON   = 1;
242
243 // Secuencia inicial LED RD0
244 for(int i = 0; i < 10; i++){
245     LATDbits.LATD0 = !LATDbits.LATD0;
246     // delay bloqueante: L = entero largo (malinterpretado - desborde)
247     for(long d = 0; d < 43000L; d++); // ~50ms a 48MHz
248 }
249 LATDbits.LATD0 = 1;
250
251 ActualizarDisplays(); // 00:00
252
253 while(1){
254
255     // ---- Tick de 500ms : (500/2.5 = 200 int.) ----
256     if(cntISR >= 200){
257         cntISR = 0;
258         MedioSecCount = !MedioSecCount;
259         LATDbits.LATD0 = !LATDbits.LATD0; // LED 1Hz
260
261         if(!is_set){
262             visDisplay = MedioSecCount;
263         } else {
264             visDisplay = 1;
265             /* ===== */
266             // Cronometro
267             if(MedioSecCount == 0){
268                 cuenta_Minutos++; // 60 seg, incrementa 1
269                 if(cuenta_Minutos >= 60){
270                     cuenta_Minutos = 0; // Reinicia el contador de segundos
271                     Minutos++;
272                     if(Minutos > 59){
273                         Minutos = 0;
274                         Hora++;
275                     }
276                     if(Hora > 23) Hora = 0; // 24 horas (0 a 23)
277                     ActualizarDisplays();
278                 }
279             }
280             // ===== */
281         }
282     }
283
284     // ---- Boton S2: incrementar ----
285     if(!PORTCbits.RC4){
286         btn_s2_cnt++;
287         if(btn_s2_cnt == 1){ // Presiona s2 flanco
288             is_set = 1;
289             visDisplay = 1;
290             IncrementarTiempo();
291         }
292         // cntISR: cada 2 interrupciones = 10ms : ~500ms (50 ticks x 10ms
293         aprox)
294         if(btn_s2_cnt > 50 && (btn_s2_cnt % 10) == 0){
295             IncrementarTiempo();
296         }
297     } else {

```

```

297         btn_s2_cnt = 0;
298     }
299
300     // ---- Boton S3: decrementar ----
301     if(!PORTCbits.RC5){
302         btn_s3_cnt++;
303         if(btn_s3_cnt == 1){
304             is_set = 1;
305             visDisplay = 1;
306             DecrementarTiempo();
307         }
308         if(btn_s3_cnt > 50 && (btn_s3_cnt % 10) == 0){
309             DecrementarTiempo();
310         }
311     } else {
312         btn_s3_cnt = 0;
313     }
314
315     // Peque a pausa no bloqueante: solo 2 interrupciones = 10ms
316     DelayXms(10);
317 }
318 return;
319 }

```

Listing 11: Estructura general del programa - Práctica 3 - Parte 2

10.0.3. Código completo parte 3

El programa en lenguaje C para la parte 3 es:

```

1  /*
2  * File:    Pr3_Codigo_Parte3.c
3  * Author:  JAIME CHIQUI
4  *
5  * Created on 31 de mayo de 2026, 16:05
6  */
7
8  // PRACTICA 3 - PARTE 3
9  // -- Funcionalidad propia: Scroll de texto "JAIME CHIQUI"
10 //    con control por pulsantes S2 y S3
11 // PIC18F4550 Configuration Bit Settings
12
13 // CONFIG1L
14 #pragma config PLLDIV = 5
15 #pragma config CPUDIV = OSC1_PLL2
16 #pragma config USBDIV = 1
17
18 // CONFIG1H
19 #pragma config FOSC = HSPLL_HS
20 #pragma config FCMEN = OFF
21 #pragma config IESO = OFF
22
23 // CONFIG2L
24 #pragma config PWRT = ON
25 #pragma config BOR = ON
26 #pragma config BORV = 3
27 #pragma config VREGEN = OFF
28
29 // CONFIG2H
30 #pragma config WDT = OFF
31 #pragma config WDTPS = 32768
32
33 // CONFIG3H

```

```

34 #pragma config CCP2MX = ON
35 #pragma config PBDEN = OFF
36 #pragma config LPT10SC = OFF
37 #pragma config MCLRE = ON
38
39 // CONFIG4L
40 #pragma config STVREN = ON
41 #pragma config LVP = OFF
42 #pragma config ICPRT = OFF
43 #pragma config XINST = OFF
44 // ** Apagar LVP para no tener corriente parasita *****
45
46 // CONFIG5L
47 #pragma config CP0 = OFF
48 #pragma config CP1 = OFF
49 #pragma config CP2 = OFF
50 #pragma config CP3 = OFF
51
52 // CONFIG5H
53 #pragma config CPB = OFF
54 #pragma config CPD = OFF
55
56 // CONFIG6L
57 #pragma config WRT0 = OFF
58 #pragma config WRT1 = OFF
59 #pragma config WRT2 = OFF
60 #pragma config WRT3 = OFF
61
62 // CONFIG6H
63 #pragma config WRTC = OFF
64 #pragma config WRTB = OFF
65 #pragma config WRTD = OFF
66
67 // CONFIG7L
68 #pragma config EBTR0 = OFF
69 #pragma config EBTR1 = OFF
70 #pragma config EBTR2 = OFF
71 #pragma config EBTR3 = OFF
72
73 // CONFIG7H
74 #pragma config EBTRB = OFF
75
76 #include <xc.h>
77
78 // =====
79 //                               VARIABLES GLOBALES
80 // =====
81 volatile char datos[4]          = {0x00, 0x00, 0x00, 0x00}; // Datos a mostrar
82                               en displays
83 volatile char multiplex          = 0; // Dígito activo 0-3
84 volatile unsigned char visDisplay = 1; // 1=mostrar 0=apagar displays
85
86 // Contador de interrupciones para medir tiempo en el main
87 volatile unsigned int cntISR     = 0;
88
89 // Estado del scroll
90 // 0 = HoLA estatico (estado inicial)
91 // 1 = Scroll corriendo
92 // 2 = Scroll en pausa (congela el cuadro actual)
93 volatile char estadoScroll = 0;
94
95 // =====
96 //                               TABLA DE CODIGOS - CATODO COMUN

```

```

96 //      Orden de bits: [a b c d e f g dp]
97 // =====
98
99 // Numeros 0-9
100 const char codigosNum[10] = {
101     0b11111100, // 0
102     0b01100000, // 1
103     0b11011010, // 2
104     0b11110010, // 3
105     0b01100110, // 4
106     0b10110110, // 5
107     0b10111110, // 6
108     0b11100000, // 7
109     0b11111110, // 8
110     0b11110110 // 9
111 };
112
113 // Letras usadas en el proyecto
114 // Orden: A b C d E F G H h i J L n o P q r S t U u y
115 const char codigosLetras[] = {
116     0b11101110, // A   ind 0
117     0b00111110, // b   ind 1
118     0b10011100, // C   ind 2
119     0b01111010, // d   ind 3
120     0b10011110, // E   ind 4
121     0b10001110, // F   ind 5
122     0b10111110, // G   ind 6
123     0b01101110, // H   ind 7
124     0b00101110, // h   ind 8
125     0b01100000, // i   ind 9
126     0b01111000, // J   ind 10
127     0b00011100, // L   ind 11
128     0b00101010, // n   ind 12
129     0b00111010, // o   ind 13
130     0b11001110, // P   ind 14
131     0b11110110, // q   ind 15
132     0b00001010, // r   ind 16
133     0b10110110, // S   ind 17
134     0b00011110, // t   ind 18
135     0b01111100, // U   ind 19
136     0b00111000, // u   ind 20
137     0b01110110 // y   ind 21
138 };
139
140 // =====
141 //      MENSAJE DE SCROLL: "JAIME CHIQUI"//
142 //      J A i M E _ C H I Q U I
143 //      Los espacios (0x00) son displays apagados
144 // =====
145 #define TAM_MSG 15
146
147 const char mensaje[TAM_MSG] = {
148     0b01111000, // J   (codigosLetras[10])
149     0b11101110, // A   (codigosLetras[0])
150     0b01100000, // i   (codigosLetras[9])
151     0b00101010, // n   <- usada para M
152     0b00101010, // n   <- usada para M
153     0b10011110, // E   (codigosLetras[4])
154     0x00, // _   espacio
155     0b10011100, // C   (codigosLetras[2])
156     0b01101110, // H   (codigosLetras[7])
157     0b01100000, // I   (1)
158     0b11100110, // Q   (codigosNum[9])

```

```

159     0b01111100,          // U   (codigosLetras[19])
160     0b01100000,          // I   (1)
161     0x00,                 // espacio final
162     0x00,                 // espacio final
163 };
164
165 // =====
166 //                               DELAYS
167 // =====
168 void Delay1ms(){
169     int veces;
170     for ( veces=0; veces <= 860; veces++ );
171 }
172
173 void DelayXms(int Milis){
174     for(int i=0; i<= Milis; i++){
175         Delay1ms();
176     }
177 }
178
179 // =====
180 // ISR ALTA PRIORIDAD - TMRO 16 Bits
181 // Interrupcion cada 2.5ms -> multiplexacion de los 4 displays
182 // Ciclo completo = 4 * 2.5ms = 10ms (100Hz, sin efecto de parpadeo)
183 // =====
184 void __interrupt(high_priority) InterrupcionHP(void){
185     if(INTCONbits.TMROIF){
186
187         // =====
188         // Recarga del TMRO para 2.5ms
189         // Fosc=48MHz Tcy=83.33ns Prescaler=1:8 Tick=666.67ns
190         // Conteos = 2.5ms / 666.67ns = 3750
191         // Carga   = 65536 - 3750 = 61786 = 0xF15A
192         // =====
193         TMROH = 0xF1;
194         TMROL = 0x5A;
195
196         // Apagar todos los digitos antes de cambiar (evita ghosting)
197         LATCbits.LATC7 = 0; // D1
198         LATCbits.LATC6 = 0; // D2
199         LATCbits.LATC1 = 0; // D3
200         LATCbits.LATC0 = 0; // D4
201
202         // Multiplexacion: mostrar u ocultar segun visDisplay
203         if(visDisplay){
204             // 0xFE = 11111110 -> limpia bit0 (dp) conservando RDO del LED
205             LATD = (datos[multiplex] & 0xFE) | (LATDbits.LATD0);
206
207             switch(multiplex){
208                 case 0: LATCbits.LATC7 = 1; break; // D1
209                 case 1: LATCbits.LATC6 = 1; break; // D2
210                 case 2: LATCbits.LATC1 = 1; break; // D3
211                 case 3: LATCbits.LATC0 = 1; break; // D4
212             }
213         } else {
214             // Displays apagados: solo preservar el estado del LED en RDO
215             LATD = 0x00 | (LATDbits.LATD0);
216         }
217
218         // Avanzar mux siempre
219         // Evita desincronizacion al volver a mostrar
220         multiplex++;
221         if(multiplex > 3) multiplex = 0;

```

```

222
223 // Contador de ticks para medir tiempos en el main
224 // Cada tick = 2.5ms -> 1000ms / 2.5ms = 400 ticks por segundo
225 cntISR++;
226
227 INTCONbits.TMROIF = 0; // Limpiar bandera de interrupcion
228 return;
229 }
230 }
231
232 // =====
233 // CARGAR VENTANA DEL SCROLL EN EL ARREGLO datos[]
234 // pos: indice del primer caracter visible en D1
235 // =====
236 void CargarVentana(int pos){
237     int i;
238     for(i = 0; i < 4; i++){
239         // Si el indice cae fuera del mensaje, mostrar espacio
240         if((pos + i) < TAM_MSG){
241             datos[i] = mensaje[pos + i];
242         } else {
243             datos[i] = 0x00; // espacio
244         }
245     }
246 }
247
248 // =====
249 // MAIN
250 // =====
251 void main(void){
252     int posScroll = 0; // Posicion actual de la ventana en el mensaje
253     int tickScroll = 0; // Acumulador de ticks para el avance del scroll
254     // Velocidad del scroll: ticks entre cada avance de 1 posicion
255     // 300ms / 2.5ms por tick = 120 ticks
256     int velocidad = 120;
257
258     // --- CONFIGURACION CRITICA PARA USAR RC4 y RC5 como GPIO ---
259     UCONbits.USBEN = 0; // Deshabilita el modulo USB
260     UCFGbits.UTRDIS = 1; // Convierte RC4/RC5 a E/S digital
261     // -----
262
263     // Configuracion de entrada / salida
264     TRISD = 0; // Puerto D todo salida (segmentos + LED)
265     TRISCbits.TRISC0 = 0; // RC0 = D4 salida
266     TRISCbits.TRISC1 = 0; // RC1 = D3 salida
267     TRISCbits.TRISC6 = 0; // RC6 = D2 salida
268     TRISCbits.TRISC7 = 0; // RC7 = D1 salida
269     TRISC |= 0b00110000; // RC4=S2 entrada, RC5=S3 entrada
270
271     // Apagar displays al inicio (evita ghosting)
272     LATD = 0;
273     LATCbits.LATC7 = 0;
274     LATCbits.LATC6 = 0;
275     LATCbits.LATC1 = 0;
276     LATCbits.LATC0 = 0;
277
278     // =====
279     // Carga inicial del TMR0 para 2.5ms
280     // =====
281     TMR0H = 0xF1;
282     TMR0L = 0x5A;
283
284     // =====

```



```

285 // Configuración de TMR0 - MODO 16 BITS, Prescaler 1:8
286 //
287 // TOCON:
288 // bit7: TMR0ON = 0 (apagado por ahora, se enciende al final)
289 // bit6: T08BIT = 0 (modo 16 bits)
290 // bit5: T0CS = 0 (reloj interno Fosc/4)
291 // bit4: T0SE = 0 (no aplica con reloj interno)
292 // bit3: PSA = 0 (prescaler activo)
293 // bit2-0: TOPS = 010 (prescaler 1:8)
294 // =====
295 TOCON = 0b00000010;
296
297 // Configuración de interrupciones con prioridades
298 RCONbits.IPEN = 1; // Habilitar sistema de prioridades
299 INTCON2bits.TMR0IP = 1; // TMR0 = alta prioridad
300 INTCONbits.TMR0IE = 1; // Habilitar interrupción de TMR0
301 INTCONbits.TMR0IF = 0; // Limpiar bandera de TMR0
302 INTCONbits.GIEH = 1; // Habilitar interrupciones globales alta prioridad
303 INTCONbits.GIEL = 1; // Habilitar interrupciones globales baja prioridad
304
305 TOCONbits.TMR0ON = 1; // Encender TMR0
306
307 // =====
308 // INICIALIZACION: RDO -> 10 pulsos de 50ms, luego ON
309 // Sirve como verificación visual de que el PIC arranca bien
310 // =====
311 for(int i = 0; i < 10; i++){
312     LATDbits.LATD0 = !LATDbits.LATD0; // Toggle LED
313     DelayXms(50);
314 }
315 LATDbits.LATD0 = 1; // LED encendido fijo
316
317 // =====
318 // ESTADO INICIAL: mostrar "HoLA" fijo hasta que se pulse S2
319 // codigosLetras[7]=H [13]=o [11]=L [0]=A
320 // =====
321 datos[0] = codigosLetras[7]; // H
322 datos[1] = codigosLetras[13]; // o
323 datos[2] = codigosLetras[11]; // L
324 datos[3] = codigosLetras[0]; // A
325 visDisplay = 1;
326 estadoScroll = 0; // Estado inicial = HoLA fijo
327
328 // =====
329 // BUCLE PRINCIPAL
330 // =====
331 while(1){
332
333     // =====
334     // LECTURA DE PULSANTES
335     // S2 (RC4): Inicia o reanuda el scroll
336     // S3 (RC5): Pausa / congela el scroll en el cuadro actual
337     // =====
338
339     // --- S2: INICIAR o REANUDAR scroll ---
340     if(PORTCbits.RC4 == 0){
341         DelayXms(20); // Antirebote
342         if(PORTCbits.RC4 == 0){
343             if(estadoScroll == 0){
344                 // Primer arranque: reiniciar posición al inicio
345                 posScroll = 0;
346                 tickScroll = 0;
347                 CargarVentana(posScroll);

```

```

348     }
349     estadoScroll = 1;           // Scroll corriendo
350     visDisplay = 1;
351     while(PORTCbits.RC4 == 0); // Esperar a soltar S2
352 }
353 }
354
355 // --- S3: PAUSAR / CONGELAR el cuadro actual ---
356 if(PORTCbits.RC5 == 0){
357     DelayXms(20);               // Antirebote
358     if(PORTCbits.RC5 == 0){
359         if(estadoScroll == 1){
360             estadoScroll = 2;   // Pausa: el cuadro queda congelado
361         } else if(estadoScroll == 2){
362             estadoScroll = 1;   // Reanudar desde donde estaba
363         } else if(estadoScroll == 0){
364             // Desde HoLA: S3 no hace nada (requiere S2 primero)
365         }
366         while(PORTCbits.RC5 == 0); // Esperar a soltar S3
367     }
368 }
369 // =====
370 // MAQUINA DE ESTADOS DEL SCROLL
371 // =====
372 if(estadoScroll == 0){
373     // -----
374     // ESTADO 0: HoLA fijo (estado inicial)
375     // Solo se sale de este estado con S2
376     // -----
377     visDisplay = 1;
378 } else if(estadoScroll == 1){
379     // -----
380     // ESTADO 1: Scroll corriendo
381     // Cada 'velocidad' ticks (= 300ms) avanza 1 posicion
382     // cntISR se incrementa en la ISR cada 2.5ms
383     // -----
384     if(cntISR >= (unsigned int)velocidad){
385         cntISR = 0;           // Resetear contador de ticks
386
387         // Cargar la ventana actual en datos[]
388         CargarVentana(posScroll);
389         posScroll++;          // Avanzar posicion para la siguiente vez
390
391         // Al llegar al final del mensaje, reiniciar (loop)
392         if(posScroll >= TAM_MSG){
393             posScroll = 0;
394         }
395     }
396 } else if(estadoScroll == 2){
397     // -----
398     // ESTADO 2: Scroll en pausa
399     // El cuadro actual permanece congelado en los displays
400     // Se reanuda con S3
401     // -----
402     cntISR = 0; // Evitar acumulacion
403 }
404 }
405 }
406 return;
407 }

```

Listing 12: Estructura general del programa - Práctica 3 - Parte 3